

Fast_Analytics_with_data.table

Quyen Di Sabino

2025-09-21

Overview

This project showcases how to perform fast, scalable analytics on large-scale transactional data using R's powerful data.table package. From exploratory data analysis to user segmentation (clustering), churn detection. This workflow is optimized for performance and insight generation.

```
##### Fast Analytics using data.table in R#####
```

```
# Load libraries
```

```
if (!require(data.table)) install.packages("data.table")
```

```
## Loading required package: data.table
```

```
library(data.table)
```

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:data.table':
```

```
##
```

```
##   between, first, last
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##   filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##   intersect, setdiff, setequal, union
```

```
# Load the CSV file
```

```
transactions_dt <- fread("transactions_dt.csv", stringsAsFactors = FALSE)
```

```
head(transactions_dt)
```

```
##   transaction_id user_id product_id price quantity      timestamp
##           <int>   <int>       <int> <num>    <int>          <POSct>
## 1:             1  881343     693641 369.05      5 2022-12-21 15:51:00
```

```
## 2:          2  177176      828777 318.07          5 2022-02-11 10:21:00
## 3:          3  419314      630281  64.49          2 2021-05-25 04:30:00
## 4:          4  369283       97469  11.86          1 2022-03-07 05:01:00
## 5:          5  832534      596512 387.25          5 2022-12-04 10:40:00
## 6:          6  529966      724783  71.40          1 2023-05-15 06:22:00
##   payment_method country
##           <char>   <char>
## 1:   Credit Card Germany
## 2:       Paypal  Brazil
## 3:       Crypto  Indian
## 4:       Crypto  Brazil
## 5:   Credit Card  Brazil
## 6:       Crypto   China
```

```
#===== Practice exercises using data.table package =====
```

```
# Add collum monetary to calculate revenue per transaction
transactions_dt[, monetary := price*quantity]
```

```
# Most popular product by sale volumm
```

```
transactions_dt[,.(total_sold = sum(price*quantity)), by = product_id][order(-total_sold)][1:10]
```

```
##   product_id total_sold
##           <int>      <num>
## 1:      89643      9540.91
## 2:     906916      9294.48
## 3:     318213      9208.57
## 4:     694813      9110.86
## 5:     674187      9077.02
## 6:     469823      9069.43
## 7:     719679      9047.27
## 8:     582593      8850.77
## 9:     859820      8840.20
## 10:    157607      8587.51
```

```
# Top 3 countries by revenue
```

```
transactions_dt[,.(revenue = sum(price*quantity)), by = country][order(-revenue)][1:3]
```

```
##   country  revenue
##     <char>    <num>
## 1:   China 126669051
## 2:     UK 126494721
## 3:     USA 126445794
```

```
# Most used payment methods globally
```

```
transactions_dt[,.(most_used_method = .N), by = payment_method][order(-most_used_method)][1:3]
```

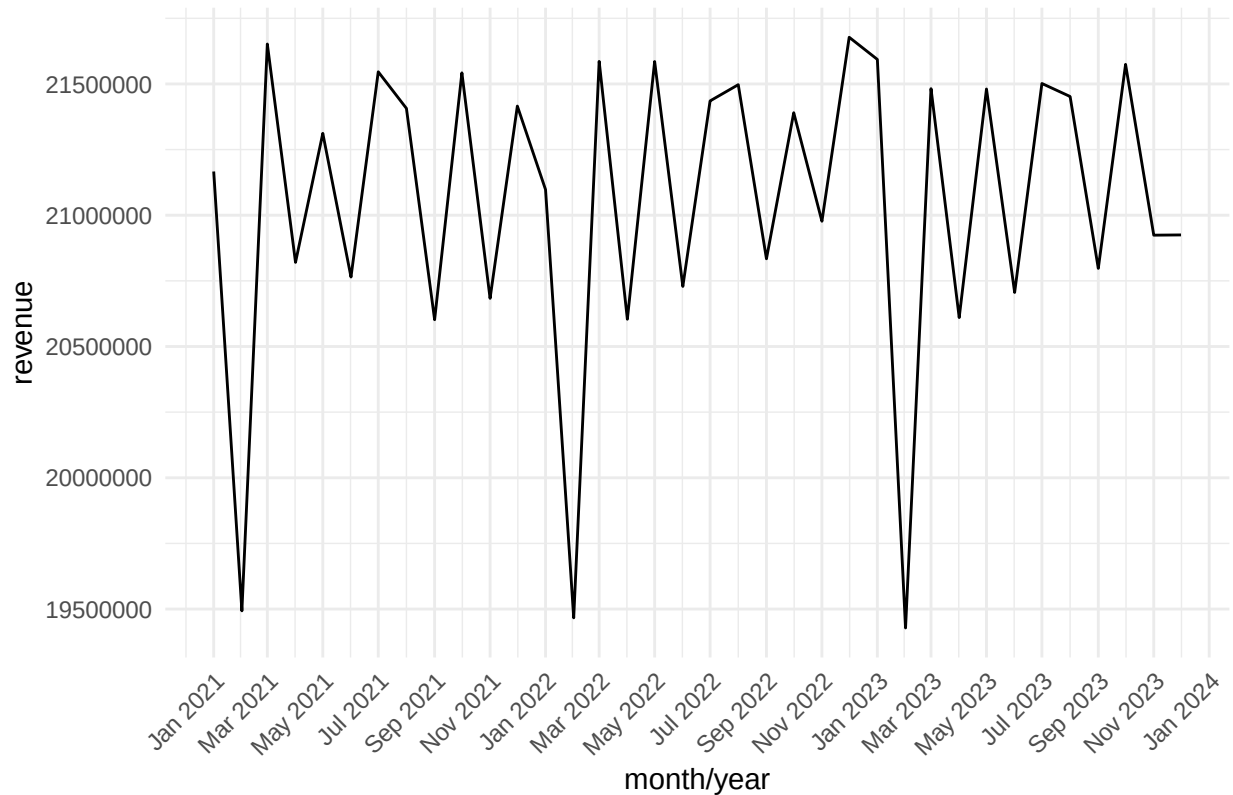
```
##   payment_method most_used_method
##           <char>          <int>
## 1:   Credit Card      250654
## 2:       Paypal      250229
## 3: Bank Transfer      250082
```

```
# Average basketsize (avg_quantity) per user
transactions_dt[, .(avg_quantity = mean(price*quantity)), by = user_id]
```

```
##           user_id avg_quantity
##           <int>         <num>
##      1:  881343         1845.2500
##      2:  177176          891.2533
##      3:  419314          459.2750
##      4:  369283          290.2533
##      5:  832534        1113.5500
##      ---
## 631820: 339691          938.4600
## 631821: 555124          847.1400
## 631822: 702787        2165.4500
## 631823: 415922           29.4300
## 631824:  30123          24.5700
```

```
# Monthly revenue trend
transactions_dt[, month := format(timestamp, "%Y-%m")]
monthly_rev <- transactions_dt[, .(revenue = sum(price*quantity)), by = month][order(month)]
# visualize
library(ggplot2)
# Trick to convert to Date-type data
ggplot(monthly_rev, aes(x = as.Date(paste0(month, "-01")), y = revenue)) +
  geom_line() +
  labs(title = "Monthly Revenue Trend", x = "month/year") +
  theme_minimal() +
  scale_x_date(date_labels = "%b %Y", date_breaks = "2 months") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Monthly Revenue Trend



Number of unique users per country

```
transactions_dt[, .(users_per_country = uniqueN(user_id)), by = country]
```

```
##      country users_per_country
##      <char>      <int>
## 1: Germany      153187
## 2: Brazil       153879
## 3: Indian       153493
## 4: China        153731
## 5: UK           153358
## 6: USA          152942
```

Peak shopping hours

```
transactions_dt[, hour := hour(timestamp)]
```

```
transactions_dt[, .(num_transactions = .N), by = hour][order(-num_transactions)][1:5]
```

```
##      hour num_transactions
##      <int>      <int>
## 1:    20         41879
## 2:     4         41852
## 3:     3         41845
## 4:     6         41809
## 5:    10         41779
```

```
# Most loyal customers
transactions_dt[, .N, by = user_id][order(-N)][1:100]
```

```
##      user_id      N
##      <int> <int>
##  1:  780524      8
##  2:  191107      8
##  3:  883244      8
##  4:   20530      8
##  5:    758      8
##  6:  208779      8
##  7:  664479      8
##  8:   41218      8
##  9:  120831      8
## 10:  374013      8
## 11:  754874      8
## 12:  427099      8
## 13:  118933      8
## 14:  617014      7
## 15:  282536      7
## 16:  118524      7
## 17:   96231      7
## 18:  909309      7
## 19:  790231      7
## 20:  672476      7
## 21:   86432      7
## 22:  305209      7
## 23:  835531      7
## 24:   93433      7
## 25:  181115      7
## 26:   48966      7
## 27:  888970      7
## 28:  914418      7
## 29:  501522      7
## 30:  711292      7
## 31:   40172      7
## 32:  409113      7
## 33:  828705      7
## 34:  888871      7
## 35:  282631      7
## 36:  444857      7
## 37:  135035      7
## 38:  186273      7
## 39:  909712      7
## 40:  749694      7
## 41:  915812      7
## 42:  473649      7
## 43:  569683      7
## 44:  538214      7
## 45:  328988      7
## 46:  486996      7
## 47:  116775      7
## 48:  975187      7
```

##	49:	530550	7
##	50:	717	7
##	51:	450648	7
##	52:	865166	7
##	53:	686257	7
##	54:	300944	7
##	55:	53739	7
##	56:	51212	7
##	57:	659470	7
##	58:	835589	7
##	59:	350468	7
##	60:	567756	7
##	61:	160020	7
##	62:	853990	7
##	63:	137382	7
##	64:	65554	7
##	65:	187245	7
##	66:	91814	7
##	67:	97273	6
##	68:	269864	6
##	69:	582464	6
##	70:	342181	6
##	71:	679520	6
##	72:	455081	6
##	73:	339856	6
##	74:	560007	6
##	75:	824141	6
##	76:	815111	6
##	77:	986399	6
##	78:	81516	6
##	79:	582541	6
##	80:	158957	6
##	81:	478952	6
##	82:	387421	6
##	83:	695724	6
##	84:	43496	6
##	85:	420667	6
##	86:	10477	6
##	87:	3069	6
##	88:	55570	6
##	89:	518931	6
##	90:	660234	6
##	91:	790877	6
##	92:	182214	6
##	93:	563823	6
##	94:	548867	6
##	95:	732414	6
##	96:	70783	6
##	97:	983318	6
##	98:	590675	6
##	99:	476912	6
##	100:	252716	6
##		user_id	N

```
# Product with highest total revenue
transactions_dt[, .(revenue = sum(monetary)), by = product_id][order(-revenue)][1:100]
```

```
##      product_id revenue
##      <int>    <num>
##  1:      89643 9540.91
##  2:      906916 9294.48
##  3:      318213 9208.57
##  4:      694813 9110.86
##  5:      674187 9077.02
##  6:      469823 9069.43
##  7:      719679 9047.27
##  8:      582593 8850.77
##  9:      859820 8840.20
## 10:      157607 8587.51
## 11:      479924 8544.93
## 12:      610789 8474.73
## 13:      320789 8452.45
## 14:      451782 8412.48
## 15:      378877 8405.32
## 16:      973047 8403.45
## 17:       65572 8303.13
## 18:      747209 8283.65
## 19:      866623 8272.48
## 20:      297869 8225.80
## 21:      270009 8217.73
## 22:      197083 8178.12
## 23:      343943 8072.34
## 24:      358058 8063.86
## 25:      761150 8051.09
## 26:      433081 8045.59
## 27:      854419 8043.53
## 28:      875514 8023.13
## 29:      657297 8018.98
## 30:      235268 7996.78
## 31:      499049 7930.01
## 32:      183576 7923.04
## 33:      812363 7918.26
## 34:      957026 7913.54
## 35:      587819 7911.06
## 36:      253840 7906.26
## 37:      803849 7846.05
## 38:      112535 7834.77
## 39:      553657 7822.67
## 40:      752773 7819.85
## 41:      259285 7786.00
## 42:      732879 7766.49
## 43:      310683 7760.98
## 44:      869750 7755.58
## 45:      392425 7747.88
## 46:      171704 7725.14
## 47:      863412 7717.26
## 48:      245583 7708.64
```

##	49:	205146	7706.74
##	50:	564096	7693.18
##	51:	152447	7673.27
##	52:	60029	7659.56
##	53:	520837	7642.92
##	54:	983737	7640.44
##	55:	555199	7637.27
##	56:	159801	7617.83
##	57:	997143	7615.33
##	58:	782993	7612.71
##	59:	569042	7600.83
##	60:	247371	7574.76
##	61:	861117	7562.44
##	62:	82868	7550.52
##	63:	63771	7549.70
##	64:	955926	7531.95
##	65:	809678	7530.55
##	66:	413580	7525.47
##	67:	836395	7511.60
##	68:	398631	7511.48
##	69:	782315	7506.96
##	70:	391651	7496.41
##	71:	258502	7484.11
##	72:	220065	7478.99
##	73:	125959	7465.45
##	74:	925485	7455.17
##	75:	471814	7454.97
##	76:	211831	7441.69
##	77:	441025	7440.87
##	78:	74464	7426.54
##	79:	734106	7425.19
##	80:	386149	7418.81
##	81:	468167	7394.98
##	82:	243423	7390.97
##	83:	701823	7378.40
##	84:	186384	7375.37
##	85:	512893	7374.29
##	86:	760773	7366.98
##	87:	756386	7339.89
##	88:	631576	7331.76
##	89:	599071	7314.35
##	90:	215919	7305.64
##	91:	930787	7287.17
##	92:	685941	7277.98
##	93:	198346	7275.32
##	94:	371860	7263.49
##	95:	377254	7247.65
##	96:	453848	7228.37
##	97:	105409	7213.01
##	98:	330619	7207.01
##	99:	146536	7167.54
##	100:	519910	7158.62
##		product_id	revenue


```
# Much faster filtering: Find all transactions in 2023 over $300
transactions_dt[year(timestamp) == 2023 & price > 300]
```

```
##      transaction_id user_id product_id price quantity      timestamp
##      <int>    <int>    <int>  <num>    <int>          <POSct>
##  1:           9  629093   185816 479.92         4 2023-05-26 02:11:00
##  2:          12  403298   800789 393.68         1 2023-04-18 05:37:00
##  3:          14  388743   264520 478.20         4 2023-04-19 14:45:00
##  4:          24  115255   199554 301.15         5 2023-12-09 20:16:00
##  5:          34  537643   952972 382.29         2 2023-07-08 15:00:00
##  ---
## 134354:      999946   36198   462372 450.04         2 2023-09-08 05:10:00
## 134355:      999951  731174   639625 331.18         3 2023-11-28 10:24:00
## 134356:      999953  266707   776114 321.85         3 2023-01-14 18:16:00
## 134357:      999958  329592   546461 411.21         3 2023-08-13 02:55:00
## 134358:      999983  663841   564037 457.06         5 2023-12-15 08:19:00
##      payment_method country monetary  month  hour
##      <char>    <char>    <num>  <char> <int>
##  1:      Crypto   Brazil  1919.68 2023-05     2
##  2:      Paypal    UK     393.68 2023-04     5
##  3:      Crypto  Germany  1912.80 2023-04    14
##  4: Bank Transfer  Indian  1505.75 2023-12    20
##  5:   Credit Card   USA    764.58 2023-07    15
##  ---
## 134354:      Crypto    UK    900.08 2023-09     5
## 134355:      Crypto  Germany  993.54 2023-11    10
## 134356:      Paypal    UK    965.55 2023-01    18
## 134357: Bank Transfer  Brazil  1233.63 2023-08     2
## 134358:   Credit Card   USA   2285.30 2023-12     8
```

```
# Load second CSV file
products_dt <- fread("products_dt.csv", stringsAsFactors = FALSE)

# Join with transactions_dt
setkey(transactions_dt, product_id) # for fast joins or lookups
setkey(products_dt, product_id)
join_data <- transactions_dt[products_dt, nomatch = 0] # inner join

# Revenue by category
join_data[, .(revenue = sum(price*quantity)), by = category]
```

```
##      category revenue
##      <char>    <num>
## 1:      Toys 1636060
## 2:      Books 1451394
## 3: Electronis 1447002
## 4:      Home 1530255
## 5:   Clothing 1516974
```

```
# ===== Cluster users based on their behavior =====
```

```

user_behavior <- transactions_dt[, .(
  total_spent = sum(price * quantity),
  total_quantity = sum(quantity),
  num_transactions = .N,
  avg_basket_value = sum(price * quantity) / .N,
  unique_products = uniqueN(product_id),
  unique_countries = uniqueN(country),
  unique_payment_methods = uniqueN(payment_method)
), by = user_id]

head(user_behavior)

```

```

##      user_id total_spent total_quantity num_transactions avg_basket_value
##      <int>      <num>         <int>          <int>          <num>
## 1:   71669      264.90           2             1           264.9000
## 2:   747851     505.65           3             1           505.6500
## 3:    25600     2356.31          9             4           589.0775
## 4:   216420     2096.60           8             3           698.8667
## 5:   190422      371.09           1             1           371.0900
## 6:   976575     2359.23           6             2          1179.6150
##      unique_products unique_countries unique_payment_methods
##              <int>          <int>          <int>
## 1:                1              1              1
## 2:                1              1              1
## 3:                4              4              2
## 4:                3              3              3
## 5:                1              1              1
## 6:                2              2              2

```

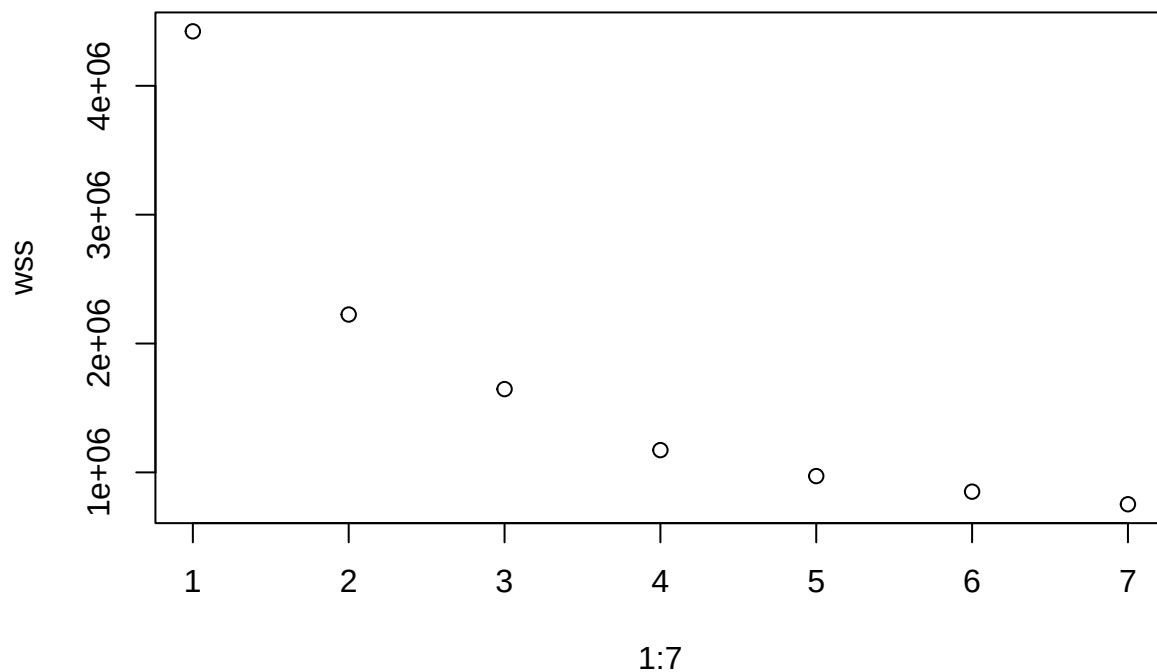
```

# Features scaling, remove user_id before scaling
user_matrix <- as.data.frame(user_behavior[, -1])
user_scaled <- scale(user_matrix)

# K-Mean Clustering
# Elbow method to find k (Can use factoextra or NbClust to find optimal k)
wss <- sapply(1:7, function(k){
  kmeans(user_scaled, centers = k, nstart = 25)$tot.withinss
})

plot(1:7, wss) # chose k = 4

```



```
set.seed(79)
kmeans_result <- kmeans(user_scaled, centers = 4, nstart = 25)

# Add cluster labels back to user_behavior data
user_behavior[, cluster := kmeans_result$cluster]

# interpret the clusters, uncovered hidden structure
cluster_summary <- user_behavior[, lapply(.SD, mean), by = cluster,
                                       .SDcols = c("total_spent", "total_quantity", "num_transactions", "avg_basket_value")]
cluster_summary
```

##	cluster	total_spent	total_quantity	num_transactions	avg_basket_value
##	<int>	<num>	<num>	<num>	<num>
## 1:	4	427.3328	2.495383	1.015063	423.4694
## 2:	2	2712.2249	10.464788	3.333974	822.8630
## 3:	3	1501.3153	6.061550	2.063432	736.8997
## 4:	1	1549.3430	4.250918	1.015071	1525.2994

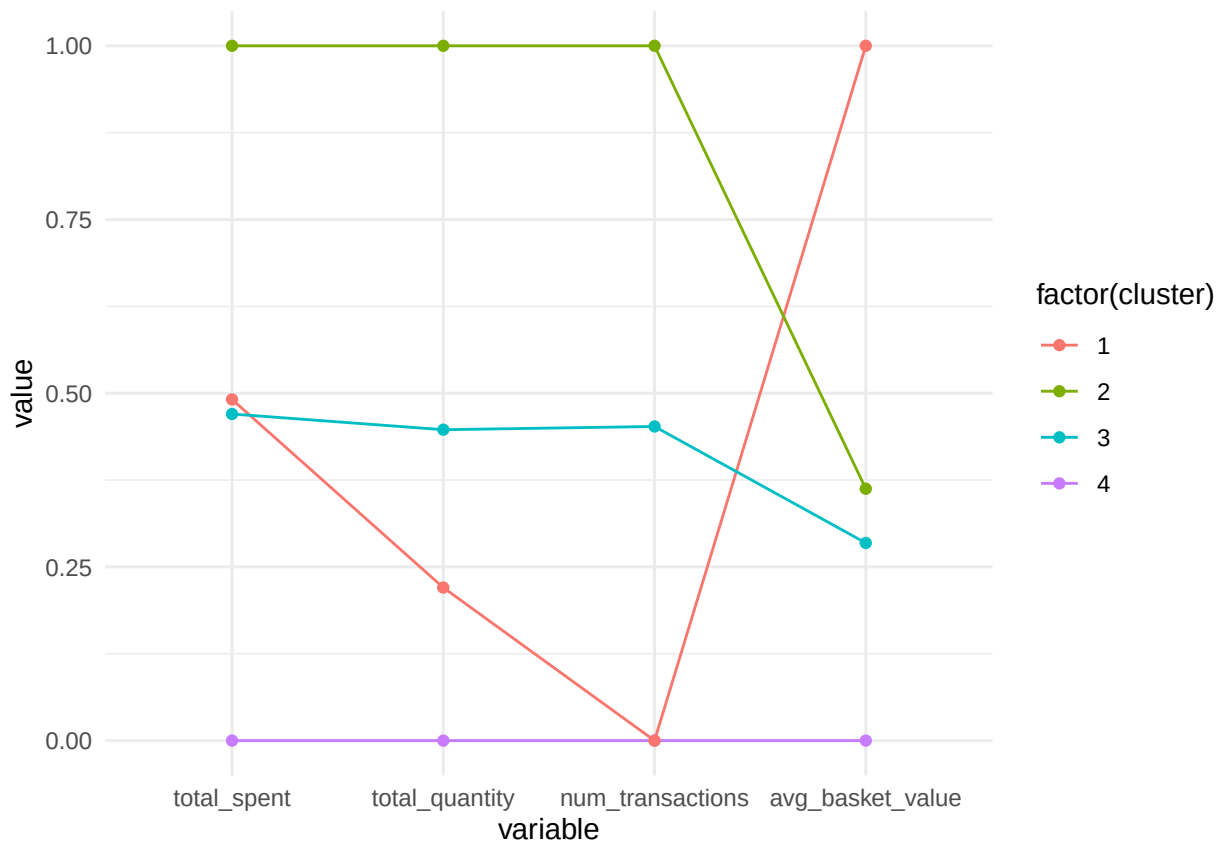
```
# Visualize the finding
# Normalize each column (min-max scaling)
normalize <- function(x) (x - min(x)) / (max(x) - min(x))
cluster_summary_norm <- cluster_summary[, lapply(.SD, normalize), .SDcols = -1]
cluster_summary_norm[, cluster := cluster_summary$cluster]
cluster_summary_norm
```

```
##      total_spent total_quantity num_transactions avg_basket_value cluster
##      <num>      <num>          <num>          <num>      <int>
## 1:  0.0000000    0.0000000    0.000000e+00    0.0000000        4
## 2:  1.0000000    1.0000000    1.000000e+00    0.3624821        2
## 3:  0.4700364    0.4474823    4.520955e-01    0.2844634        3
## 4:  0.4910561    0.2202843    3.512080e-06    1.0000000        1
```

```
# data.table::melt() to long format
```

```
cluster_long <- melt(cluster_summary_norm, id.vars = "cluster")
```

```
ggplot(cluster_long, aes(x = variable, y = value, group = factor(cluster), color = factor(cluster))) +
  geom_line() +
  geom_point() +
  theme_minimal()
```



```
# Cluster behavior interpretation:
```

```
# Cluster 1 High avg_basket_value, but low transaction count → "Premium Buyers"
```

```
# Cluster 2 Highest total spent, quantity, and transactions → "Power Users"
```

```
# Cluster 3 Moderate across all features → "Engaged Users"
```

```
# Cluster 4 Very few transactions, low spend → "Low-Value Users"
```

```
# Add cluster labels to transactions_dt for later use
```

```
transactions_dt <- merge(transactions_dt, user_behavior[, .(user_id, cluster)],
  by = "user_id", all.x = TRUE) # left join
```

```
# Group by cluster
transactions_dt[, .N, by = cluster]
```

```
##      cluster      N
##      <int> <int>
## 1:      1 114502
## 2:      4 264034
## 3:      2 228894
## 4:      3 392570
```

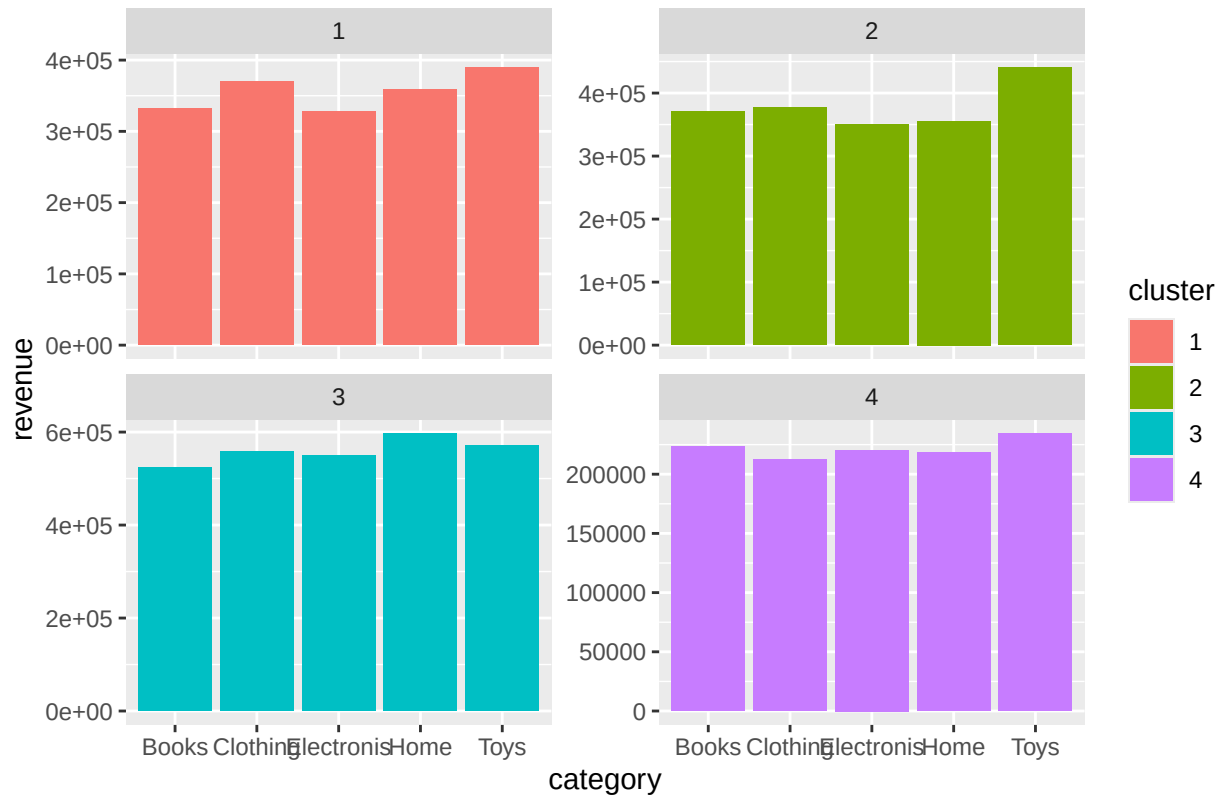
```
# Distribution by cluster and country
transactions_dt[, .N, by = .(cluster, country)]
```

```
##      cluster country      N
##      <int> <char> <int>
## 1:      1      UK 19144
## 2:      4 Germany 44074
## 3:      4      USA 43658
## 4:      2   China 38318
## 5:      2      UK 38472
## 6:      2  Indian 37929
## 7:      4   China 44109
## 8:      4  Indian 44292
## 9:      2      USA 38140
## 10:      2 Brazil 38065
## 11:      4 Brazil 44017
## 12:      3   China 65481
## 13:      3  Indian 65505
## 14:      3 Brazil 65699
## 15:      3 Germany 65390
## 16:      1 Germany 18951
## 17:      3      USA 65387
## 18:      4      UK 43884
## 19:      1      USA 19044
## 20:      2 Germany 37970
## 21:      3      UK 65108
## 22:      1   China 19133
## 23:      1  Indian 18898
## 24:      1 Brazil 19332
##      cluster country      N
```

```
# Add cluster labels to join_data
join_data <- merge(join_data, user_behavior[, .(user_id, cluster)],
                  by = "user_id", all.x = TRUE)
# Top product-categories by cluster
top_categories <- join_data[, .(revenue = sum(price*quantity)), by = .(cluster, category)]
top_categories <- top_categories[, cluster := factor(cluster)]

ggplot(top_categories, aes(category, revenue, fill = cluster)) +
  geom_bar(stat = "identity") +
  facet_wrap(~ cluster, scale = "free_y") +
  labs(title = "Categories by Revenue per Cluster")
```

Categories by Revenue per Cluster



```
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

```
## List of 136
## $ line                                :List of 6
## ..$ colour                          : chr "black"
## ..$ linewidth                       : num 0.5
## ..$ linetype                        : num 1
## ..$ lineend                         : chr "butt"
## ..$ arrow                           : logi FALSE
## ..$ inherit.blank                  : logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_line" "element"
## $ rect                               :List of 5
## ..$ fill                            : chr "white"
## ..$ colour                          : chr "black"
## ..$ linewidth                       : num 0.5
## ..$ linetype                        : num 1
## ..$ inherit.blank                  : logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_rect" "element"
## $ text                               :List of 11
## ..$ family                          : chr ""
## ..$ face                            : chr "plain"
## ..$ colour                          : chr "black"
## ..$ size                            : num 11
## ..$ hjust                           : num 0.5
```

```

## ..$ vjust          : num 0.5
## ..$ angle          : num 0
## ..$ lineheight     : num 0.9
## ..$ margin         : 'margin' num [1:4] 0points 0points 0points 0points
## .. ..- attr(*, "unit")= int 8
## ..$ debug          : logi FALSE
## ..$ inherit.blank: logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ title            : NULL
## $ aspect.ratio     : NULL
## $ axis.title        : NULL
## $ axis.title.x      :List of 11
## ..$ family         : NULL
## ..$ face            : NULL
## ..$ colour         : NULL
## ..$ size           : NULL
## ..$ hjust          : NULL
## ..$ vjust          : num 1
## ..$ angle          : NULL
## ..$ lineheight     : NULL
## ..$ margin         : 'margin' num [1:4] 2.75points 0points 0points 0points
## .. ..- attr(*, "unit")= int 8
## ..$ debug          : NULL
## ..$ inherit.blank: logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ axis.title.x.top  :List of 11
## ..$ family         : NULL
## ..$ face            : NULL
## ..$ colour         : NULL
## ..$ size           : NULL
## ..$ hjust          : NULL
## ..$ vjust          : num 0
## ..$ angle          : NULL
## ..$ lineheight     : NULL
## ..$ margin         : 'margin' num [1:4] 0points 0points 2.75points 0points
## .. ..- attr(*, "unit")= int 8
## ..$ debug          : NULL
## ..$ inherit.blank: logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ axis.title.x.bottom : NULL
## $ axis.title.y      :List of 11
## ..$ family         : NULL
## ..$ face            : NULL
## ..$ colour         : NULL
## ..$ size           : NULL
## ..$ hjust          : NULL
## ..$ vjust          : num 1
## ..$ angle          : num 90
## ..$ lineheight     : NULL
## ..$ margin         : 'margin' num [1:4] 0points 2.75points 0points 0points
## .. ..- attr(*, "unit")= int 8
## ..$ debug          : NULL
## ..$ inherit.blank: logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_text" "element"

```

```

## $ axis.title.y.left          : NULL
## $ axis.title.y.right        :List of 11
## ..$ family                  : NULL
## ..$ face                     : NULL
## ..$ colour                   : NULL
## ..$ size                     : NULL
## ..$ hjust                    : NULL
## ..$ vjust                    : num 1
## ..$ angle                    : num -90
## ..$ lineheight               : NULL
## ..$ margin                   : 'margin' num [1:4] 0points 0points 0points 2.75points
## .. ..- attr(*, "unit")= int 8
## ..$ debug                    : NULL
## ..$ inherit.blank: logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ axis.text                  :List of 11
## ..$ family                   : NULL
## ..$ face                     : NULL
## ..$ colour                   : chr "grey30"
## ..$ size                     : 'rel' num 0.8
## ..$ hjust                    : NULL
## ..$ vjust                    : NULL
## ..$ angle                    : NULL
## ..$ lineheight               : NULL
## ..$ margin                   : NULL
## ..$ debug                    : NULL
## ..$ inherit.blank: logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ axis.text.x                :List of 11
## ..$ family                   : NULL
## ..$ face                     : NULL
## ..$ colour                   : NULL
## ..$ size                     : NULL
## ..$ hjust                    : num 1
## ..$ vjust                    : num 1
## ..$ angle                    : num 45
## ..$ lineheight               : NULL
## ..$ margin                   : 'margin' num [1:4] 2.2points 0points 0points 0points
## .. ..- attr(*, "unit")= int 8
## ..$ debug                    : NULL
## ..$ inherit.blank: logi FALSE
## ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ axis.text.x.top            :List of 11
## ..$ family                   : NULL
## ..$ face                     : NULL
## ..$ colour                   : NULL
## ..$ size                     : NULL
## ..$ hjust                    : NULL
## ..$ vjust                    : num 0
## ..$ angle                    : NULL
## ..$ lineheight               : NULL
## ..$ margin                   : 'margin' num [1:4] 0points 0points 2.2points 0points
## .. ..- attr(*, "unit")= int 8
## ..$ debug                    : NULL

```



```

## ..$ inherit.blank: logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ axis.text.x.bottom : NULL
## $ axis.text.y :List of 11
## ..$ family : NULL
## ..$ face : NULL
## ..$ colour : NULL
## ..$ size : NULL
## ..$ hjust : num 1
## ..$ vjust : NULL
## ..$ angle : NULL
## ..$ lineheight : NULL
## ..$ margin : 'margin' num [1:4] 0points 2.2points 0points 0points
## ..- attr(*, "unit")= int 8
## ..$ debug : NULL
## ..$ inherit.blank: logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ axis.text.y.left : NULL
## $ axis.text.y.right :List of 11
## ..$ family : NULL
## ..$ face : NULL
## ..$ colour : NULL
## ..$ size : NULL
## ..$ hjust : num 0
## ..$ vjust : NULL
## ..$ angle : NULL
## ..$ lineheight : NULL
## ..$ margin : 'margin' num [1:4] 0points 0points 0points 2.2points
## ..- attr(*, "unit")= int 8
## ..$ debug : NULL
## ..$ inherit.blank: logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ axis.text.theta : NULL
## $ axis.text.r :List of 11
## ..$ family : NULL
## ..$ face : NULL
## ..$ colour : NULL
## ..$ size : NULL
## ..$ hjust : num 0.5
## ..$ vjust : NULL
## ..$ angle : NULL
## ..$ lineheight : NULL
## ..$ margin : 'margin' num [1:4] 0points 2.2points 0points 2.2points
## ..- attr(*, "unit")= int 8
## ..$ debug : NULL
## ..$ inherit.blank: logi TRUE
## ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ axis.ticks : list()
## ..- attr(*, "class")= chr [1:2] "element_blank" "element"
## $ axis.ticks.x : NULL
## $ axis.ticks.x.top : NULL
## $ axis.ticks.x.bottom : NULL
## $ axis.ticks.y : NULL
## $ axis.ticks.y.left : NULL

```

```

## $ axis.ticks.y.right      : NULL
## $ axis.ticks.theta       : NULL
## $ axis.ticks.r           : NULL
## $ axis.minor.ticks.x.top  : NULL
## $ axis.minor.ticks.x.bottom : NULL
## $ axis.minor.ticks.y.left : NULL
## $ axis.minor.ticks.y.right : NULL
## $ axis.minor.ticks.theta  : NULL
## $ axis.minor.ticks.r      : NULL
## $ axis.ticks.length       : 'simpleUnit' num 2.75points
##   .- attr(*, "unit")= int 8
## $ axis.ticks.length.x     : NULL
## $ axis.ticks.length.x.top : NULL
## $ axis.ticks.length.x.bottom : NULL
## $ axis.ticks.length.y     : NULL
## $ axis.ticks.length.y.left : NULL
## $ axis.ticks.length.y.right : NULL
## $ axis.ticks.length.theta : NULL
## $ axis.ticks.length.r     : NULL
## $ axis.minor.ticks.length : 'rel' num 0.75
## $ axis.minor.ticks.length.x : NULL
## $ axis.minor.ticks.length.x.top : NULL
## $ axis.minor.ticks.length.x.bottom : NULL
## $ axis.minor.ticks.length.y : NULL
## $ axis.minor.ticks.length.y.left : NULL
## $ axis.minor.ticks.length.y.right : NULL
## $ axis.minor.ticks.length.theta : NULL
## $ axis.minor.ticks.length.r : NULL
## $ axis.line               : list()
##   .- attr(*, "class")= chr [1:2] "element_blank" "element"
## $ axis.line.x             : NULL
## $ axis.line.x.top         : NULL
## $ axis.line.x.bottom      : NULL
## $ axis.line.y             : NULL
## $ axis.line.y.left        : NULL
## $ axis.line.y.right       : NULL
## $ axis.line.theta         : NULL
## $ axis.line.r             : NULL
## $ legend.background       : list()
##   .- attr(*, "class")= chr [1:2] "element_blank" "element"
## $ legend.margin           : 'margin' num [1:4] 5.5points 5.5points 5.5points 5.5points
##   .- attr(*, "unit")= int 8
## $ legend.spacing          : 'simpleUnit' num 11points
##   .- attr(*, "unit")= int 8
## $ legend.spacing.x        : NULL
## $ legend.spacing.y        : NULL
## $ legend.key              : list()
##   .- attr(*, "class")= chr [1:2] "element_blank" "element"
## $ legend.key.size         : 'simpleUnit' num 1.2lines
##   .- attr(*, "unit")= int 3
## $ legend.key.height       : NULL
## $ legend.key.width        : NULL
## $ legend.key.spacing      : 'simpleUnit' num 5.5points
##   .- attr(*, "unit")= int 8

```

```

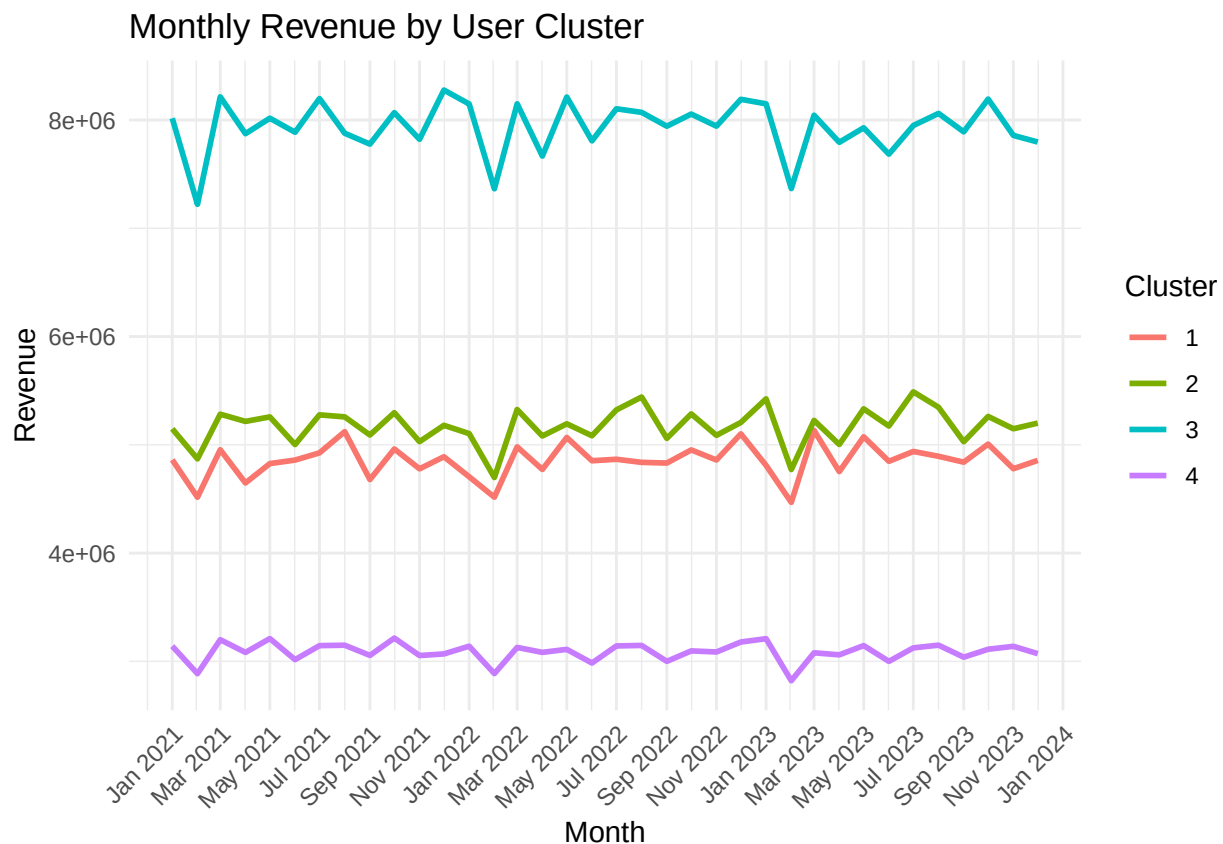
## $ legend.key.spacing.x      : NULL
## $ legend.key.spacing.y      : NULL
## $ legend.frame              : NULL
## $ legend.ticks              : NULL
## $ legend.ticks.length       : 'rel' num 0.2
## $ legend.axis.line          : NULL
## $ legend.text                :List of 11
##   ..$ family                : NULL
##   ..$ face                   : NULL
##   ..$ colour                 : NULL
##   ..$ size                   : 'rel' num 0.8
##   ..$ hjust                  : NULL
##   ..$ vjust                  : NULL
##   ..$ angle                  : NULL
##   ..$ lineheight             : NULL
##   ..$ margin                 : NULL
##   ..$ debug                  : NULL
##   ..$ inherit.blank: logi TRUE
##   ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ legend.text.position       : NULL
## $ legend.title               :List of 11
##   ..$ family                : NULL
##   ..$ face                   : NULL
##   ..$ colour                 : NULL
##   ..$ size                   : NULL
##   ..$ hjust                  : num 0
##   ..$ vjust                  : NULL
##   ..$ angle                  : NULL
##   ..$ lineheight             : NULL
##   ..$ margin                 : NULL
##   ..$ debug                  : NULL
##   ..$ inherit.blank: logi TRUE
##   ..- attr(*, "class")= chr [1:2] "element_text" "element"
## $ legend.title.position      : NULL
## $ legend.position            : chr "right"
## $ legend.position.inside     : NULL
## $ legend.direction           : NULL
## $ legend.byrow               : NULL
## $ legend.justification       : chr "center"
## $ legend.justification.top   : NULL
## $ legend.justification.bottom : NULL
## $ legend.justification.left  : NULL
## $ legend.justification.right  : NULL
## $ legend.justification.inside : NULL
## $ legend.location            : NULL
## $ legend.box                 : NULL
## $ legend.box.just            : NULL
## $ legend.box.margin          : 'margin' num [1:4] 0cm 0cm 0cm 0cm
##   ..- attr(*, "unit")= int 1
## $ legend.box.background      : list()
##   ..- attr(*, "class")= chr [1:2] "element_blank" "element"
## $ legend.box.spacing         : 'simpleUnit' num 11points
##   ..- attr(*, "unit")= int 8
## [list output truncated]

```

```
## - attr(*, "class")= chr [1:2] "theme" "gg"
## - attr(*, "complete")= logi TRUE
## - attr(*, "validate")= logi TRUE

# Time series per cluster, by month
monthly_cluster_rev <- transactions_dt[, .(revenue = sum(price*quantity)), by = .(cluster, month)]
monthly_cluster_rev[, month := as.Date(paste0(month, "-01"))] # Convert month to proper Date for plotting

ggplot(monthly_cluster_rev, aes(x = month, y = revenue, color = as.factor(cluster))) +
  geom_line(size = 1) +
  labs(
    title = "Monthly Revenue by User Cluster",
    x = "Month",
    y = "Revenue",
    color = "Cluster"
  ) +
  theme_minimal() +
  scale_x_date(date_labels = "%b %Y", date_breaks = "2 months") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

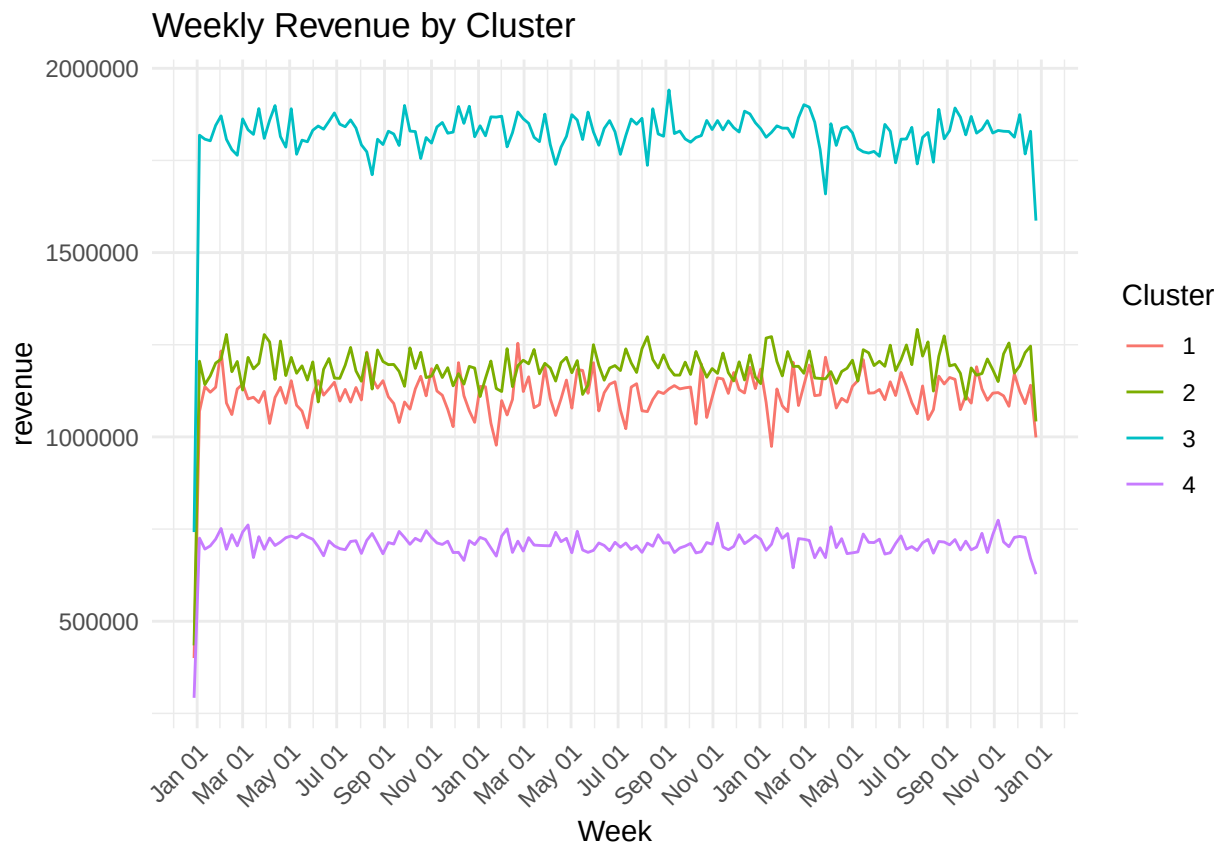


```
# More time series per cluster, by week
transactions_dt[, week := as.Date(cut(timestamp, breaks = "week"))]
weekly_rev <- transactions_dt[, .(revenue = sum(price * quantity)), by = .(cluster, week)]
ggplot(weekly_rev, aes(x = week, y = revenue, color = as.factor(cluster))) +
  geom_line() +
```

```

scale_x_date(date_labels = "%b %d", date_breaks = "2 month") +
labs(
  title = "Weekly Revenue by Cluster",
  x = "Week",
  color = "Cluster"
) +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



```

# More time series per cluster, by quarter
if(!require(zoo)) install.packages("zoo") # for as.yearqtr()

## Loading required package: zoo

##
## Attaching package: 'zoo'

## The following objects are masked from 'package:data.table':
##
##   yearmon, yearqtr

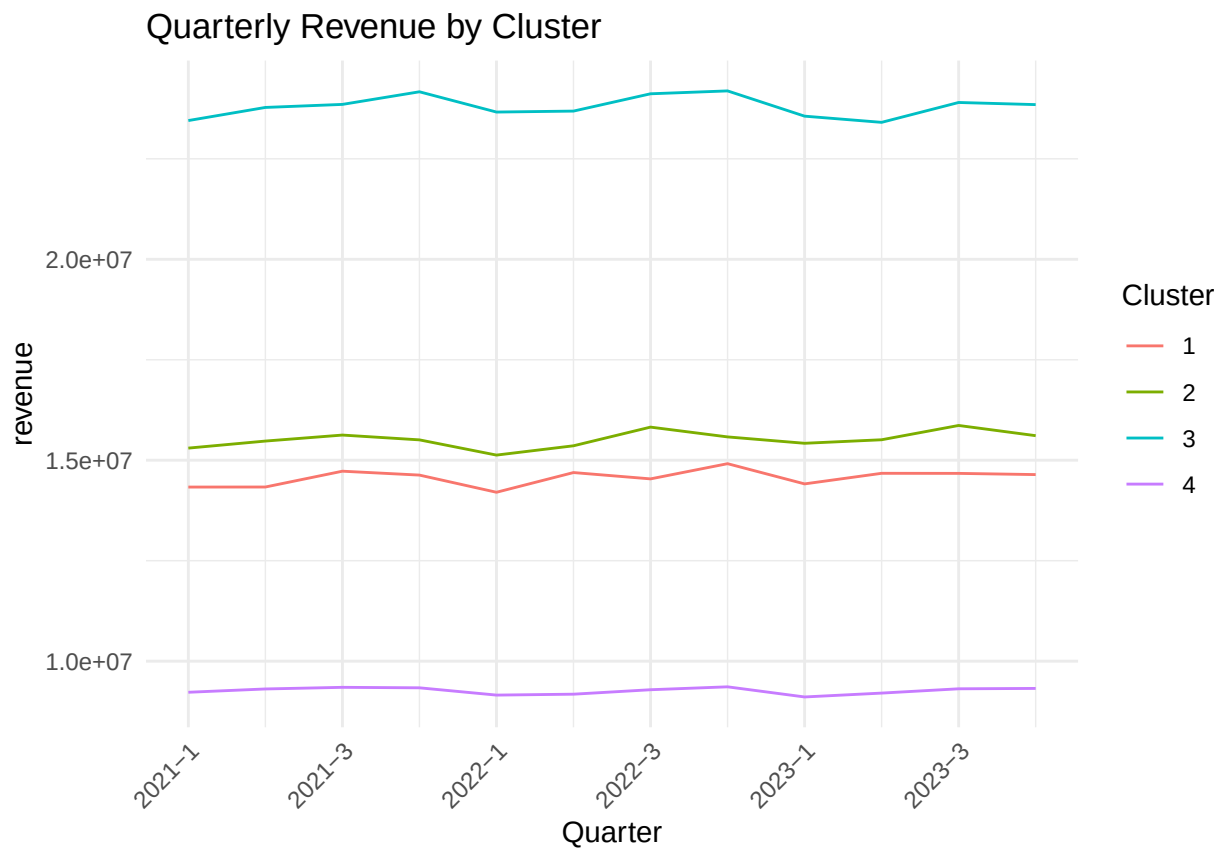
## The following objects are masked from 'package:base':
##
##   as.Date, as.Date.numeric

```

```

transactions_dt[, quarter := as.yearqtr(timestamp)]
quarterly_rev <- transactions_dt[, .(revenue = sum(price * quantity)), by = .(cluster, quarter)]
ggplot(quarterly_rev, aes(x = quarter, y = revenue, color = as.factor(cluster))) +
  geom_line() +
  labs(
    title = "Quarterly Revenue by Cluster",
    x = "Quarter",
    color = "Cluster"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



```

===== Churn Analysis =====

# Sort transaction by user and time to use data.table::shift()
setorder(transactions_dt, user_id, timestamp)

# Calculate time gaps between purchases per user
transactions_dt[, days_since_last_purchase := as.numeric(
  difftime(timestamp, shift(timestamp), units = "days")
), by = user_id]

# Get last purchase date per user
last_purchase_dt <- transactions_dt[, .(last_purchase = max(timestamp)), by = user_id]

```

```

# Calculate days since last purchase per user
today <- as.POSIXct("2023-12-31")
last_purchase_dt[, days_since_last := as.numeric(difftime(today, last_purchase, units = "days"))]

# Flag users as churned if inactivate for > 60 days
churn_threshold <- 60
last_purchase_dt[, churn_flag := days_since_last > churn_threshold]

last_purchase_dt[, .N, by = churn_flag]

##      churn_flag      N
##      <lgcl>  <int>
## 1:      TRUE 578312
## 2:     FALSE 53512

# Merge churn status back into user behavior data
user_behavior <- merge(user_behavior, last_purchase_dt, by = "user_id", all.x = TRUE)

ggplot(last_purchase_dt, aes(x = days_since_last, fill = churn_flag)) +
  geom_histogram(binwidth = 10, color = "white", alpha = 0.8) +
  labs(title = "Distribution of Days Since Last Purchase", x = "Days Since Last", y = "User Count") +
  theme_minimal()

```

